

Smart Pointers — Ինչո՞ւ են պետք

C++ Հարցազրույցի ամբողջական ուղեցույց

Այս ուղեցույցը բացատրում է, ինչո՞ւ են smart pointer-ները պետք՝ raw pointer-ի խնդիրները, RAII, երեք տեսակները, Rule of Zero/Three/Five, օրինակներով և հարցազրույցի հարցերով:

1. Raw Pointer-ի խնդիրները

Smart pointer-ները լուծում են raw pointer-ի 4 մեծ խնդիր:

1.1 Memory Leak

```
void f() {  
    int* p = new int(5);  
    if (error) return;    // LEAK -- delete չի հասնի  
    delete p;  
}
```

1.2 Dangling Pointer

```
int* p = new int(5);  
delete p;  
cout << *p;    // DANGLING -- ջնջված memory (UB)
```

1.3 Double Delete

```
int* p = new int(5);  
delete p;  
delete p;    // DOUBLE DELETE -- UB / crash
```

1.4 Ownership-ի անպարզություն

```
void process(int* p);    // ո՞վ է պատասխանատու delete-ի? անհայտ
```

Raw pointer-ից հայտնի չէ՝ ով է owner-ը, ով պետք է delete անի -> սխալներ:

2. Լուծումը — Smart Pointers (RAII)

Smart pointer-ը object է, որ **ավտոմատ կառավարում է memory-ն** (RAII)՝ scope-ի վերջում ինքն է delete անում:

```
#include <memory>
void f() {
    auto p = std::make_unique<int>(5);
    if (error) return;    // ՉԿԱ leak -- ավտոմատ delete
}                        // scope-ի վերջ -> ավտոմատ delete
```

raw pointer: ձեռքով delete, leak/dangling/double-delete վտանգ
smart pointer: ավտոմատ delete (RAII), ownership պարզ, չկա leak

3. Երեք տեսակները

unique_ptr -> EXCLUSIVE ownership (մեկ owner), zero overhead, move-only
shared_ptr -> SHARED ownership (մի քանի owner), reference counting
weak_ptr -> NON-OWNING (չի ավելացնում count), circular reference-ի լուծում

unique_ptr

```
auto u = std::make_unique<int>(5);    // մեկ owner
// copy չկա, միայն move
```

shared_ptr

```
auto s1 = std::make_shared<int>(5);   // count 1
auto s2 = s1;                          // count 2 (shared)
```

weak_ptr

```
std::weak_ptr<int> w = s1;           // չի ավելացնում count -- circular խուսափում
```

4. Որ մեկը երբ

```
unique_ptr  -> DEFAULT. մեկ owner, exclusive resource (90% դեպքերում)
shared_ptr  -> իսկապես մի քանի owner պետք է (shared data)
weak_ptr    -> shared_ptr-ի հետ, circular reference կոտրելու / observer
```

Կանոն. **սկսիր unique_ptr-ից**; անցիր shared_ptr միայն երբ իսկապես shared ownership պետք է:

5. Rule of Zero / Three / Five

Rule of Three (հին C++)

Եթե class-ը պետք ունի destructor, copy constructor, կամ copy assignment-ից **մեկը** (սովորաբար raw pointer-ի պատճառով), պետք ունի **երեքն** էլ:

```
class Bad {
    int* data;
public:
    Bad() { data = new int[100]; }
    ~Bad() { delete[] data; }           // 1
    Bad(const Bad&);                   // 2 (deep copy)
    Bad& operator=(const Bad&);       // 3
};
```

Rule of Five (C++11)

- move constructor + move assignment (5 ընդամենը):

Rule of Zero (ԼԱՎԱԳՈՒՅՆ)

Եթե օգտագործում ես smart pointer / STL container -> **ՈՉ ՄԻ** ձեռքով special member պետք չէ:

```
class Good {  
    std::unique_ptr<int[]> data;    // RAII ինքնին  
    std::vector<int> v;  
    std::string s;  
};    // ոչ մի dtor/copy/move պետք չէ -- compiler-ի default-ները ճիշտ են
```

6. Ինչու smart pointers — ամփոփ

- + Ավտոմատ memory management (RAII) -> չկա leak
- + Չկա dangling / double-delete
- + Պարզ ownership (unique = մեկ, shared = քանի)
- + Exception-safe (scope-ի վերջ -> ավտոմատ delete)
- + Rule of Zero -> քիչ կոդ, քիչ սխալ
- shared_ptr overhead (atomic count) -> օգտագործիր պետք եղածին

7. Հարցազրույցի հարցեր և պատասխաններ

Ինչո՞ւ smart pointers:

Raw pointer-ի խնդիրների (leak, dangling, double-delete, անպարզ ownership) լուծում RAII-ով -> ավտոմատ memory management:

Երեք տեսակները:

unique_ptr (exclusive), shared_ptr (shared, ref count), weak_ptr (non-owning, circular լուծում):

Որ մեկը default:

unique_ptr -- zero overhead, exclusive ownership. shared միայն երբ իսկապես shared պետք է:

Rule of Zero ի՞նչ է:

RAII տիպեր (smart pointer/container) օգտագործելիս ոչ մի ձեռքով dtor/copy/move պետք չէ:

Rule of Three/Five:

Three (dtor/copy-ctor/copy-assign) հին; Five (+move ctor/assign) C++11:

shared_ptr overhead:

Atomic reference count + control block -> unique-ից ծանր; օգտագործիր միայն երբ shared պետք է:

Circular reference ինչպես ես լուծում:

weak_ptr (չի ավելացնում count) մեկ ուղղությամբ:

Ամփոփում (Cheat Sheet)

RAW POINTER-ի խնդիրներ: leak | dangling | double-delete | անպարզ ownership

SMART POINTERS (RAII -> ավտոմատ delete):

unique_ptr -> exclusive (մեկ owner), zero overhead, DEFAULT

shared_ptr -> shared (մի քանի), reference counting

weak_ptr -> non-owning, circular reference լուծում

RULE OF:

Three -> dtor + copy-ctor + copy-assign (հին)

Five -> + move-ctor + move-assign (C++11)

Zero -> RAII տիպեր -> ՈՉ ՄԻ ձեռքով (ԼԱՎԱԳՈՒՅՆ)

ԿԱՆՈՆ: սկսիր unique_ptr-ից; shared միայն երբ իսկապես shared ownership պետք է